UNIVERSITY OF ENGINEERING & TECHNOLOGY

Peshawar, Pakistan

Lab Report

The For Loop & Nested For Loop

STUDENT NAME	**Shahzad Gul**
REGISTRATION NO	**0671**
SUBJECT	**Computer Programming (EE-170L)**
LAB NUMBER	**06**
SEMESTER	**Spring 2026**

SPRING 2026 SESSION

06

Table of Contents

1. Objective

2. Theory

2.1 The for Loop

2.2 Flow Control in a for Loop

2.3 Nested for Loop

3. Lab Tasks and Solutions

Task 1: Numbers from 1 to 10

Task 2: Multiplication Table

Task 3: Sum using for Loop

Task 4: Triangle Pattern

4. Conclusion

1. Objective

The objectives of this laboratory session are to understand and apply the fundamental concepts of iterative control structures in C++ programming. Specifically, this lab focuses on:

- Understanding the **for loop** syntax, execution flow, and practical applications
- Implementing **nested for loops** for multi-dimensional iteration and pattern generation
- Writing C++ programs that efficiently handle repetitive tasks using loop constructs

2. Theory

2.1 The for Loop

A `for` loop is a repetition control structure that allows the programmer to efficiently write a loop that needs to execute a specific number of times. The for loop is particularly useful when a value runs from a starting point to an ending point with a constant increment or decrement.

The for loop is preferred over the `while` loop when the number of iterations is known beforehand, as it consolidates the initialization, condition checking, and update operations into a single line.

Syntax of for Loop

```
for (initialization; condition; update)
{
    // statements to be executed
}
```

Component	Description	Execution
Initialization	Declares and initializes the loop control variable	Executed only once at the beginning
Condition	Boolean expression evaluated before each iteration	If true, loop body executes; if false, loop terminates
Update	Modifies the loop control variable (increment/decrement)	Executed after each iteration of the loop body

2.2 Flow Control in a for Loop

The execution of a for loop follows a precise sequence of steps:

1. **Initialization:** The loop control variable is initialized. This step is performed exactly once when the loop begins.
2. **Condition Evaluation:** The condition expression is evaluated. If the condition evaluates to `true`, the loop body is executed. If `false`, the loop terminates and control passes to

the next statement after the loop.

3. **Loop Body Execution:** The statements inside the loop body are executed.
4. **Update:** The loop control variable is updated (incremented or decremented). Control then returns to step 2 for condition re-evaluation.
5. **Termination:** When the condition becomes `false`, the for loop terminates.

Example 1: Program to print values from 5 to 1 using a for loop.

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 5; i > 0; i--)
    {
        cout << i << endl;
    }
    cout << "Outside the loop now";
    return 0;
}
```

```
OUTPUT
5
4
3
2
1
Outside the loop now
```

Example 2: Program to check if a number is prime using a for loop.

```
#include <iostream>
using namespace std;

int main()
{
    int num;
    cout << "Enter a number please \n";
    cin >> num;
    int flag = 0;
    for (int i = 2; i < num; i++)
    {
        if (num % i == 0)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 1)
    {
        cout << num << " is not a prime number \n";
    }
    else
    {
        cout << num << " is a prime number \n";
    }
    return 0;
}
```

In this example, the program reads an integer from the user and checks whether it is divisible by any number from 2 to `num-1`. If a divisor is found, the `flag` is set to 1 and the loop exits using `break`. After the loop, the flag value determines whether the number is prime or not.

2.3 Nested for Loop

A **nested for loop** is a for loop placed inside another for loop. For each iteration of the outer loop, the inner loop completes all of its iterations. Nested loops are commonly used for processing multi-dimensional data structures, generating patterns, and implementing algorithms that require multiple levels of iteration.

Syntax of Nested for Loop

```
for (initialization; condition; update)      // Outer loop
{
    for (initialization; condition; update)  // Inner loop
    {
        // inner loop statements
    }
    // outer loop statements
}
```

Example: Program to print a descending number pattern.

```
#include <iostream>
using namespace std;

int main()
{
    for (int num = 7; num > 0; num--)
    {
        for (int j = 1; j <= num; j++)  // nested loop
        {
            cout << j << "  ";
        }
        cout << "\n";
    }
    return 0;
}
```

```
OUTPUT
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

In this example, the outer loop controls the number of elements to print in each row (decreasing from 7 to 1), while the inner loop prints the numbers from 1 up to the current value of `num`.

3. Lab Tasks and Solutions

Task 1: Display Numbers from 1 to 10

Problem Statement

Write a C++ program that uses a for loop to display numbers from 1 to 10, with each number on a separate line.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Numbers from 1 to 10:" << endl;
    for (int i = 1; i <= 10; i++)
    {
        cout << i << endl;
    }
    return 0;
}
```

EXPECTED OUTPUT

```
Numbers from 1 to 10:
1
2
3
4
5
6
7
8
9
10
```

Explanation: The loop initializes `i` to 1, checks if `i <= 10`, and increments `i` by 1 after each iteration. The loop executes 10 times, printing the value of `i` on each iteration.

Task 2: Multiplication Table

Problem Statement

Write a C++ program that generates a multiplication table from 1 to 10 using nested for loops. Display the table in a formatted manner.

Solution

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << "Multiplication Table (1 to 10)" << endl;
    cout << "================================" << endl;

    for (int i = 1; i <= 10; i++)
    {
        for (int j = 1; j <= 10; j++)
        {
            cout << setw(4) << i * j;
        }
        cout << endl;
    }
    return 0;
}
```

```
EXPECTED OUTPUT
Multiplication Table (1 to 10)
================================
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
```

Explanation: The outer loop iterates through rows (1 to 10), and the inner loop iterates through columns (1 to 10). The product `i * j` is printed with a fixed width of 4 characters using `setw(4)` for proper alignment. After each inner loop completes, a newline is printed to move to the next row.

Task 3: Sum Using for Loop

> **Problem Statement**
>
> Write a C++ program that prompts the user to enter a positive integer, then uses a for loop to calculate the sum of all numbers from 1 to the entered integer.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    int sum = 0;

    cout << "Enter a positive integer: ";
    cin >> n;

    for (int i = 1; i <= n; i++)
    {
        sum = sum + i;
    }

    cout << "The sum of numbers from 1 to " << n << " is: " << sum << endl;
    return 0;
}
```

```
SAMPLE OUTPUT
Enter a positive integer: 5
The sum of numbers from 1 to 5 is: 15
```

Explanation: The program reads a positive integer `n` from the user. The for loop iterates from 1 to `n`, adding each value to the accumulator variable `sum`. The final sum is displayed after the loop completes. For input 5, the calculation is: 1 + 2 + 3 + 4 + 5 = 15.

Task 4: Triangle Pattern Printing

Problem Statement

Write a C++ program that uses nested for loops to print a triangle pattern of asterisks. Each row should have an increasing number of asterisks, and the number of rows should be determined by user input.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    int rows;

    cout << "Enter the number of rows: ";
    cin >> rows;

    for (int i = 1; i <= rows; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

SAMPLE OUTPUT (INPUT: 5)

```
Enter the number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

Explanation: The outer loop controls the row number (`i` from 1 to `rows`). The inner loop prints asterisks, with the number of asterisks in each row equal to the current row number `i`. After each row is printed, a newline moves the cursor to the next line. This creates a right-aligned triangular pattern with an increasing number of asterisks per row.

4. Conclusion

In this laboratory session, the concepts of the `for` loop and nested `for` loop were thoroughly explored through theoretical study and practical implementation. The key findings and learning outcomes are summarized below:

- The **for loop** provides a compact and efficient mechanism for executing a block of code a predetermined number of times, with initialization, condition checking, and variable update consolidated into a single statement.
- **Nested for loops** enable multi-dimensional iteration patterns essential for matrix operations, pattern generation, and table construction.
- Proper use of the `break` statement allows early termination of loops when a specific condition is met, improving program efficiency.
- All four lab tasks were successfully implemented, demonstrating proficiency in applying loop constructs to solve computational problems.

The skills acquired in this lab form a foundational basis for more advanced programming concepts including array processing, algorithm design, and data structure manipulation in subsequent coursework.